

SMARTFENIX

Documentación Técnica del Sistema

Proyecto:	SmartFenix API REST
Framework:	Spring Boot 3.2.5
Lenguaje:	Java 21
Base de datos:	MySQL 8.0 (Docker)
Puerto API:	8099
Puerto MySQL:	3308
Fecha:	Mayo 2026

Documento generado para el proyecto académico SmartFenix.

Incluye análisis E-R, diseño de base de datos, configuración y ejemplos de uso con Postman.

Índice de Contenidos

1. Análisis y Diagrama Entidad-Relación	3
2. Diseño de la Base de Datos	4
3. Conexión con la Base de Datos en Spring Boot	5
4. Alta de Registros (Endpoints POST)	6
5. Visualización de Datos (Endpoints GET)	7
6. Gestión de Registros (Endpoints PUT y DELETE)	8

1. Análisis y Diagrama Entidad-Relación

El proyecto SmartFenix gestiona cuatro entidades principales: **Cliente**, **Proyecto**, **Empleado** y **Tarea**. A continuación se describe cada entidad y las relaciones entre ellas.

1.1 Entidades del sistema

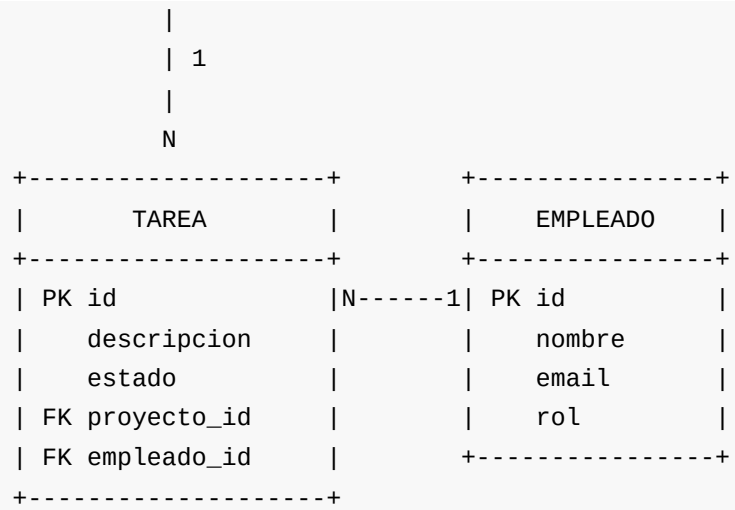
Entidad	Descripción	Campos principales
Cliente	Empresa o persona que contrata proyectos a SmartFenix	id, nombre, telefono, empresa
Proyecto	Trabajo asignado a un cliente con fechas de inicio y fin	id, nombre, fechaInicio, fechaFin, cliente_id
Empleado	Trabajador interno de SmartFenix con rol asignado	id, nombre, email, rol
Tarea	Unidad de trabajo dentro de un proyecto, asignada a un empleado	id, descripcion, estado, proyecto_id, empleado_id

1.2 Relaciones entre entidades

Relación	Tipo	Descripción
Cliente → Proyecto	1 : N	Un cliente puede tener varios proyectos. Un proyecto pertenece a un único cliente.
Proyecto → Tarea	1 : N	Un proyecto puede tener varias tareas. Una tarea pertenece a un único proyecto.
Empleado → Tarea	1 : N	Un empleado puede tener asignadas varias tareas. Cada tarea tiene un único responsable.

1.3 Diagrama E-R (representación textual)

+-----+		+-----+
CLIENTE		PROYECTO
+-----+		+-----+
PK id	1-----N	PK id
nombre		nombre
telefono		fecha_inicio
empresa		fecha_fin
+-----+		FK cliente_id
		+-----+

**Nota sobre las cardinalities:**

Las anotaciones JPA utilizadas son `@ManyToOne` en las entidades Proyecto y Tarea, con `@JoinColumn` para definir la columna de clave foránea en la base de datos.

2. Diseño de la Base de Datos

Hibernate genera automáticamente las tablas en MySQL a partir de las anotaciones JPA definidas en las clases del paquete `domain`. La configuración `ddl-auto=create` recrea las tablas en cada arranque de la aplicación.

2.1 Tabla: `cliente`

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Identificador único del cliente
nombre	VARCHAR(255)	NOT NULL	Nombre del cliente (obligatorio)
telefono	VARCHAR(255)	–	Teléfono de contacto (opcional)
empresa	VARCHAR(255)	–	Nombre de la empresa (opcional)

2.2 Tabla: `proyecto`

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Identificador único del proyecto
nombre	VARCHAR(255)	NOT NULL	Nombre del proyecto (obligatorio)
fecha_inicio	DATE	–	Fecha de inicio del proyecto
fecha_fin	DATE	–	Fecha de finalización prevista
cliente_id	BIGINT	FOREIGN KEY → cliente(id)	Cliente propietario del proyecto

2.3 Tabla: `empleado`

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Identificador único del empleado
nombre	VARCHAR(255)	NOT NULL	Nombre completo (obligatorio)

Columna	Tipo	Restricciones	Descripción
email	VARCHAR(255)	NOT NULL, UNIQUE	Email corporativo (único en la BD)
rol	VARCHAR(255)	–	Rol o cargo del empleado (opcional)

2.4 Tabla: `tarea`

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Identificador único de la tarea
descripcion	VARCHAR(255)	NOT NULL	Descripción de la tarea (obligatorio)
estado	VARCHAR(255)	–	Estado: Pendiente / En progreso / Completada
proyecto_id	BIGINT	FOREIGN KEY → proyecto(id)	Proyecto al que pertenece la tarea
empleado_id	BIGINT	FOREIGN KEY → empleado(id)	Empleado responsable de la tarea

2.5 Restricciones destacadas

Entidad	Campo	Restricción	Motivo
Cliente	nombre	NOT NULL	Anotación <code>@NotBlank</code> en la entidad Java
Empleado	email	NOT NULL, UNIQUE	Anotación <code>@Email + unique=true</code>
Proyecto	cliente_id	FOREIGN KEY	Relación <code>@ManyToOne</code> con Cliente
Tarea	proyecto_id	FOREIGN KEY	Relación <code>@ManyToOne</code> con Proyecto
Tarea	empleado_id	FOREIGN KEY	Relación <code>@ManyToOne</code> con Empleado

3. Conexión con la Base de Datos en Spring Boot

Spring Boot conecta con MySQL a través del fichero `application.properties`. El contenedor Docker expone MySQL en el puerto **3308** del host (mapeado al 3306 interno del contenedor).

3.1 Fichero `application.properties`

Ubicación: `src/main/resources/application.properties`

```
spring.datasource.url=jdbc:mysql://localhost:3308/smartfenix_db?
createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=root1234
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=create
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
server.port=8099
```

3.2 Explicación de cada propiedad

Propiedad	Valor	Explicación
<code>datasource.url</code>	<code>jdbc:mysql://localhost:3308/smartfenix_db</code>	Dirección del servidor MySQL. El parámetro <code>createDatabaseIfNotExist=true</code> crea la base de datos automáticamente si no existe.
<code>datasource.username</code>	<code>root</code>	Usuario de MySQL configurado en Docker.
<code>datasource.password</code>	<code>root1234</code>	Contraseña del usuario root definida en la variable de entorno <code>MYSQL_ROOT_PASSWORD</code> del contenedor.
<code>driver-class-name</code>	<code>com.mysql.cj.jdbc.Driver</code>	Driver JDBC para MySQL 8.x (incluido en la dependencia <code>mysql-connector-j</code> del pom.xml).
<code>ddl-auto</code>	<code>create</code>	Hibernate elimina y recrea las tablas en cada arranque. Útil en desarrollo; en producción se usa <i>update</i> o <i>validate</i> .

Propiedad	Valor	Explicación
show-sql	true	Imprime en consola las sentencias SQL generadas por Hibernate.
server.port	8099	Puerto en el que la API REST escucha peticiones HTTP.

3.3 Docker Compose (compose.yaml)

Se levanta con el comando `docker compose up -d` desde la raíz del proyecto:

```
version: '3.8'
services:
  mysql_server:
    image: mysql:8.0
    container_name: mysql_smartfenix_new
    ports:
      - "3308:3306"
    environment:
      - MYSQL_ROOT_PASSWORD=root1234
      - MYSQL_DATABASE=smartfenix_db
    command: --default-authentication-plugin=mysql_native_password
    volumes:
      - ./mysql_data:/var/lib/mysql

  phpmyadmin_web:
    image: phpmyadmin/phpmyadmin:latest
    container_name: phpmyadmin_smartfenix_new
    ports:
      - "8098:80"
    environment:
      - PMA_HOST=mysql_server
      - PMA_PORT=3306
      - PMA_USER=root
      - PMA_PASSWORD=root1234
    depends_on:
      - mysql_server
```

Acceso a phpMyAdmin:

Una vez levantado Docker, se puede administrar la base de datos visualmente en `http://localhost:8098` con el usuario `root` y contraseña `root1234`.

4. Alta de Registros — Endpoints POST

Los endpoints POST permiten crear nuevos registros en la base de datos. Se debe enviar un JSON en el cuerpo de la petición con la cabecera `Content-Type: application/json`. La respuesta exitosa devuelve el objeto creado con código HTTP **201 Created**.

4.1 Crear Cliente

URL: POST `http://localhost:8099/api/clientes`

Campos obligatorios: nombre

Body (JSON):

```
{
  "nombre": "Empresa Gamma",
  "telefono": "666112233",
  "empresa": "Gamma Digital SL"
}
```

Respuesta 201:

```
{
  "id": 3,
  "nombre": "Empresa Gamma",
  "telefono": "666112233",
  "empresa": "Gamma Digital SL"
}
```

4.2 Crear Empleado

URL: POST `http://localhost:8099/api/empleados`

Campos obligatorios: nombre, email (debe ser un email válido y único)

```
{
  "nombre": "Luis Fernández",
  "email": "luis.fernandez@smartfenix.com",
  "rol": "Arquitecto de Software"
}
```

4.3 Crear Proyecto

URL: POST `http://localhost:8099/api/proyectos`

Campos obligatorios: nombre. El campo `cliente` debe referenciar un cliente existente.

```
{
  "nombre": "Plataforma E-Learning",
  "fechaInicio": "2026-06-01",
  "fechaFin": "2026-12-31",
  "cliente": { "id": 1 }
}
```

4.4 Crear Tarea

URL: POST `http://localhost:8099/api/tareas`

Campos obligatorios: descripcion. Los campos `proyecto` y `empleado` deben referenciar registros existentes.

```
{
  "descripcion": "Desarrollar módulo de login OAuth2",
  "estado": "Pendiente",
  "proyecto": { "id": 1 },
  "empleado": { "id": 2 }
}
```

Configuración en Postman:

Seleccionar método POST, introducir la URL y en la pestaña

Body

seleccionar

raw → **JSON**

. Pegar el JSON del ejemplo y pulsar Send.

5. Visualización de Datos — Endpoints GET

Los endpoints GET permiten consultar los registros almacenados en la base de datos. No requieren body en la petición. Devuelven **200 OK** con un JSON, o **404 Not Found** si el recurso solicitado no existe.

5.1 Listado de endpoints GET

URL	Descripción	Respuesta
GET /api/clientes	Lista todos los clientes	Array JSON · 200 OK
GET /api/clientes/{id}	Obtiene un cliente por su ID	Objeto JSON · 200 / 404
GET /api/empleados	Lista todos los empleados	Array JSON · 200 OK
GET /api/empleados/{id}	Obtiene un empleado por su ID	Objeto JSON · 200 / 404
GET /api/proyectos	Lista todos los proyectos	Array JSON · 200 OK
GET /api/proyectos/{id}	Obtiene un proyecto por su ID	Objeto JSON · 200 / 404
GET /api/tareas	Lista todas las tareas	Array JSON · 200 OK
GET /api/tareas/{id}	Obtiene una tarea por su ID	Objeto JSON · 200 / 404

5.2 Ejemplo: GET /api/clientes

```
[
  {
    "id": 1,
    "nombre": "Tech Global",
    "telefono": "600123456",
    "empresa": "Tech Global Corp"
  },
  {
    "id": 2,
    "nombre": "Innova Solutions",
    "telefono": "600987654",
    "empresa": "Innova SL"
  }
]
```

5.3 Ejemplo: GET /api/tareas/1

```
{
  "id": 1,
  "descripcion": "Configurar base de datos AWS",
  "estado": "En progreso",
  "proyecto": {
    "id": 1,
    "nombre": "Migración Cloud",
    "fechaInicio": "2026-04-27",
    "fechaFin": "2026-06-06",
    "cliente": {
      "id": 1,
      "nombre": "Tech Global",
      "telefono": "600123456",
      "empresa": "Tech Global Corp"
    }
  },
  "empleado": {
    "id": 1,
    "nombre": "Ana García",
    "email": "ana.garcia@smartfenix.com",
    "rol": "Desarrolladora Backend"
  }
}
```

Datos iniciales (DataLoader):

Al arrancar la aplicación, si la base de datos está vacía, se insertan automáticamente 2 clientes, 2 proyectos, 3 empleados y 3 tareas de ejemplo. Estos datos están disponibles inmediatamente para probar los endpoints GET sin necesidad de crear registros manualmente.

6. Gestión de Registros — Endpoints PUT y DELETE

6.1 Actualizar registros — PUT

El método PUT recibe el **ID del registro en la URL** y el **objeto completo actualizado en el body**. Devuelve el objeto modificado con código **200 OK**, o **404 Not Found** si no existe.

Actualizar Cliente — PUT /api/clientes/{id}

Ejemplo: actualizar el cliente con ID = 1

```
URL: PUT http://localhost:8099/api/clientes/1

Body:
{
  "nombre": "Tech Global Internacional",
  "telefono": "699000111",
  "empresa": "Tech Global Corp Internacional"
}
```

Actualizar Empleado — PUT /api/empleados/{id}

```
URL: PUT http://localhost:8099/api/empleados/2

Body:
{
  "nombre": "Carlos López",
  "email": "carlos.lopez@smartfenix.com",
  "rol": "Director de Operaciones"
}
```

Actualizar Proyecto — PUT /api/proyectos/{id}

```
URL: PUT http://localhost:8099/api/proyectos/1

Body:
{
  "nombre": "Migración Cloud v2",
  "fechaInicio": "2026-05-01",
  "fechaFin": "2026-07-31",
  "cliente": { "id": 1 }
}
```

Actualizar Tarea — PUT /api/tareas/{id}

```

URL: PUT http://localhost:8099/api/tareas/1

Body:
{
  "descripcion": "Configurar base de datos AWS",
  "estado": "Completada",
  "proyecto": { "id": 1 },
  "empleado": { "id": 1 }
}

```

6.2 Eliminar registros — DELETE

El método DELETE recibe únicamente el **ID en la URL**, sin body. Devuelve **204 No Content** si se elimina correctamente, o **404 Not Found** si el registro no existe.

URL	Respuesta éxito	Respuesta error
DELETE /api/clientes/{id}	204 No Content	404 Not Found
DELETE /api/proyectos/{id}	204 No Content	404 Not Found
DELETE /api/empleados/{id}	204 No Content	404 Not Found
DELETE /api/tareas/{id}	204 No Content	404 Not Found

Ejemplo DELETE en Postman

```

Método: DELETE
URL:    http://localhost:8099/api/tareas/3
Body:   (vacío, no se necesita)

Respuesta si existe:    HTTP 204 No Content  (body vacío)
Respuesta si no existe: HTTP 404 Not Found

```

Orden recomendado para eliminar sin errores de integridad referencial:

1. Eliminar primero las
Tareas
(dependen de Proyecto y Empleado)
2. Eliminar los
Proyectos
(dependen de Cliente)
3. Eliminar los
Cientes

y

Empleados

(sin dependencias)

6.3 Resumen completo de todos los endpoints

Entidad	GET todos	GET por ID	POST	PUT	DELETE
Cliente	/api/clientes	/api/clientes/{id}	✓	✓	✓
Proyecto	/api/proyectos	/api/proyectos/{id}	✓	✓	✓
Empleado	/api/empleados	/api/empleados/{id}	✓	✓	✓
Tarea	/api/tareas	/api/tareas/{id}	✓	✓	✓